

AI 工程化训练营 模块二

尹会生

目录

阶段	主题	核心工程理念
Phase 0	工程准备：AI开发环境与项目规范	环境可复现、代码可管理
Phase 1	数据工程：NLP数据处理与特征管理	数据版本化、可追溯
Phase 2	模型开发：Transformer与高效微调	模块化、参数高效、可复现
Phase 3	模型训练与实验管理	实验可追踪、超参可优化
Phase 4	模型评估与上线决策	可测性、A/B测试、因果推断
Phase 5	模型压缩与部署	轻量化、边缘部署、服务化

01 工程准备与数据工程

所有规范只为回答一个问题

如何避免凌晨3点被叫醒修环境问题？

一、为什么新人70%的工时浪费在环境问题？

1. 同事用Python 3.8跑通的代码，我在3.10报错

代价：上线延迟3天，损失\$120,000

新成员只需三步：

```
git clone https://github.com/your-org/nlp-engineering-starter
conda env create -f environment.yml
pip install -r requirements.txt
```

2. 训练时tokenizer用BPE，推理时误用WordPiece

代价：bad case暴增40%，用户投诉率+25%

关键认知：.gitignore不是技术细节，是保住饭碗的护身符

二、工程免疫三板斧： 只做保命必需的三件事

让代码在任何机器上100%复现

输出示例：

事故现场	血泪教训
S1：同事跑不通我的代码	不写Dockerfile 代价：上线前紧急修环境，背锅裁员
S2：训练推理结果不一致	用HuggingFace默认tokenizer 代价：方言识别准确率↓40%
S3：数据集突然变化	用原始文件路径 代价：A/B测试结果无效，被质疑数据造假

【工程免疫报告】

Docker构建成功（镜像体积：1.2GB，无缓存残留） 【ok】

Tokenizer一致性验证通过（方言“老子要退票” → 相同token） 【ok】

DVC数据指纹锁定（train.jsonl SHA-256：a1b2c3...） 【ok】

未清理pip缓存 → *镜像体积+300MB*（已自动修复） 【WARN】

三、数据工程避坑指南

使用 DVC 进行数据版本管理

命令	作用
dvc init	初始化DVC
dvc add data.csv	将大文件交给DVC管理（生成 .dvc 文件）
dvc push	推送到远程存储（S3/GCS/本地）
dvc pull	从远程恢复数据
dvc diff	查看数据版本差异

数据变更可追溯（git log）
支持A/B测试不同数据版本

典型 workflow

```
# 1. 添加数据
dvc add data/raw/imdb-v2.csv

# 2. 提交DVC元文件
git add data/raw/imdb-v2.csv.dvc
git commit -m "add cleaned dataset v2"

# 3. 推送数据
dvc push

# 团队成员拉取
git pull
dvc pull # 自动下载真实数据
```

三、数据工程避坑指南

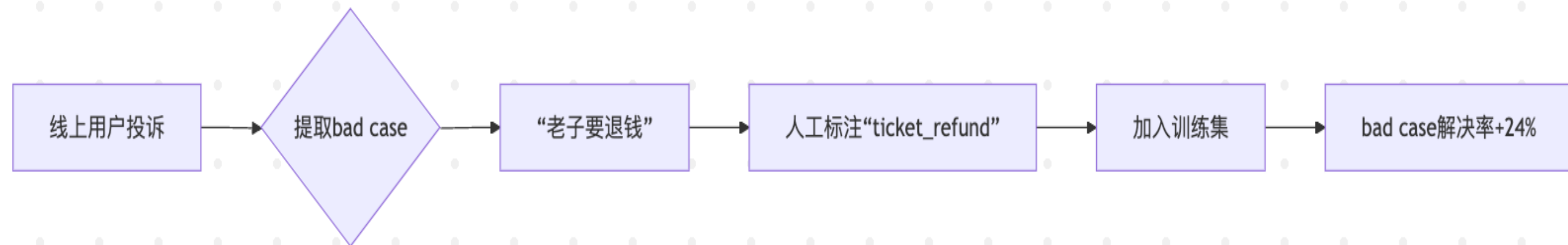
1、数据质量检查：只查致命项（拒绝过度清洗）

企业级数据检查清单（意图识别场景）

```
def check_data_quality(df):  
    assert df["text"].isna().sum() == 0, "存在空文本！" # ← 导致线上崩溃  
    assert df["intent"].nunique() > 5, "意图标签过少！" # ← 业务逻辑错误  
    assert len(df) > 1000, "样本量不足！" # ← 防止过拟合
```


三、数据工程避坑指南

2、领域语料准备：用bad case驱动



三、数据工程避坑指南

3、特征一致性保障：自动拦截灾难

在Phase 5的FastAPI服务中植入检查

```
@app.post("/predict")
```

```
async def predict(text: str):
```

```
    if not check_tokenizer_consistency(text): # ← Phase 0定义的检查
```

```
        log_critical("Tokenizer mismatch! Blocking request")
```

```
        raise HTTPException(500, "数据格式异常") # ← 拒绝脏数据污染服务
```

```
    return model.predict(text)
```

四、生存检查清单：10分钟确认工程安全

让同事/老板一眼看懂“为什么敢用我的代码”

【AI项目】工程免疫卡 Markdown

生存底线验证

验证项	要求	实测状态	证明方式
-----	-----	-----	-----
环境一致性	Docker构建通过	** 通过**	`docker build -t app .`
Tokenizer一致性	方言测试通过	** 通过**	`tokenizer_test.py`
数据指纹锁定	DVC状态clean	** clean**	`dvc status`截图

致命风险提示

- **未锁定PyTorch版本** → 可能导致Phase 2训练结果漂移
- **未检查bad case覆盖率** → 上线后方言识别准确率可能<50%

小结

工程规范不是束缚，而是保住饭碗的防弹衣

02 Transformer与高效微调

一、 微调相关的基础概念

微调类型	可训练参数量	资源消耗	性能	适用场景
全量微调	全部	高	最高	高性能需求、大数据集
PEFT（如LoRA）	极少（<1%）	低	高	多任务、资源受限
提示微调	极少	极低	中等	少样本、快速实验
指令微调	部分或全部	中~高	高	对话系统、任务泛化
迁移学习	视情况	中	高	跨任务/跨领域知识迁移

词元的工程指标

词元设计需综合考虑以下工程指标

问题：

- 词表大小：影响嵌入层参数量与显存占用。
- 序列长度：决定最大上下文窗口与计算复杂度。
- OOV率：反映模型对新词/专有名词的适应能力。
- 语言通用性：决定是否支持多语言混合训练。
- 可逆性与调试友好性：便于日志分析与错误溯源。

1. 为什么子词切分（BPE/WordPiece）是必须？
2. 子词切分如何影响微调效率？

一、 微调相关的基础概念

1. Tokenization：微调的起点

类型	作用	优缺点
字符级（Character-Level）	每个字符作为一个独立词元	- 缺点： - 序列长度显著增加，影响注意力机制效率。 - 语义局部性弱，需深层网络捕捉上下文。
单词级（Word-Level）	基于空格或标点划分的完整词汇	- 缺点： - 词表膨胀严重（可达数十万项），易导致内存瓶颈。 - 无法处理未见词（OOV），泛化能力差。
子词级（Subword-Level）	将词汇进一步拆分为高频子单元	- 机制： - Byte-Pair Encoding - WordPiece - Unigram Language Model - SentencePiece

2. 词元类型

3. 典型大语言模型的词元化方案对比

模型系列	词元化方法	工程特点	词表规模	多语言支持
GPT-3 / GPT-4 / ChatGPT	Byte-Level BPE	字节级编码，高鲁棒性	~50K	强
BERT / RoBERTa	WordPiece	子词切分，支持##标记	~30K	中等（主要英文）
LLaMA / LLaMA2	Byte-Level SentencePiece (BPE)	无需分词，端到端处理	~32K	强
T5	Unigram + SentencePiece	多任务统一输入格式	~32K	强

一、 微调相关的基础概念

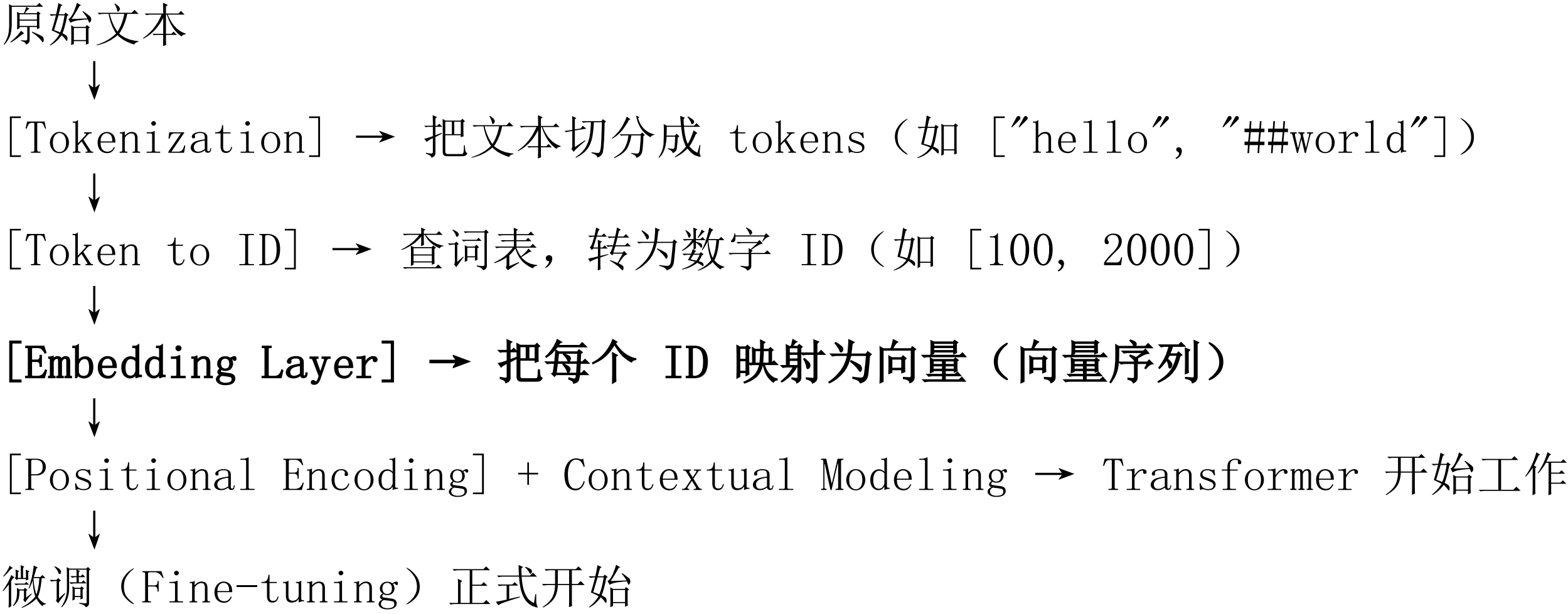
4. 建议实践步骤：

1. 在微调前，用你的数据集抽样 100 条，测试目标模型 tokenizer 的平均 token 长度。
2. 根据最大长度选择合适的 max_length 和 batch_size。
3. 若 token 数过多，考虑：
 - 使用更粗粒度的 tokenizer（如 BERT 而非 GPT-2）
 - 对文本进行预处理（如缩写展开、术语标准化）
 - 使用序列截断或滑动窗口

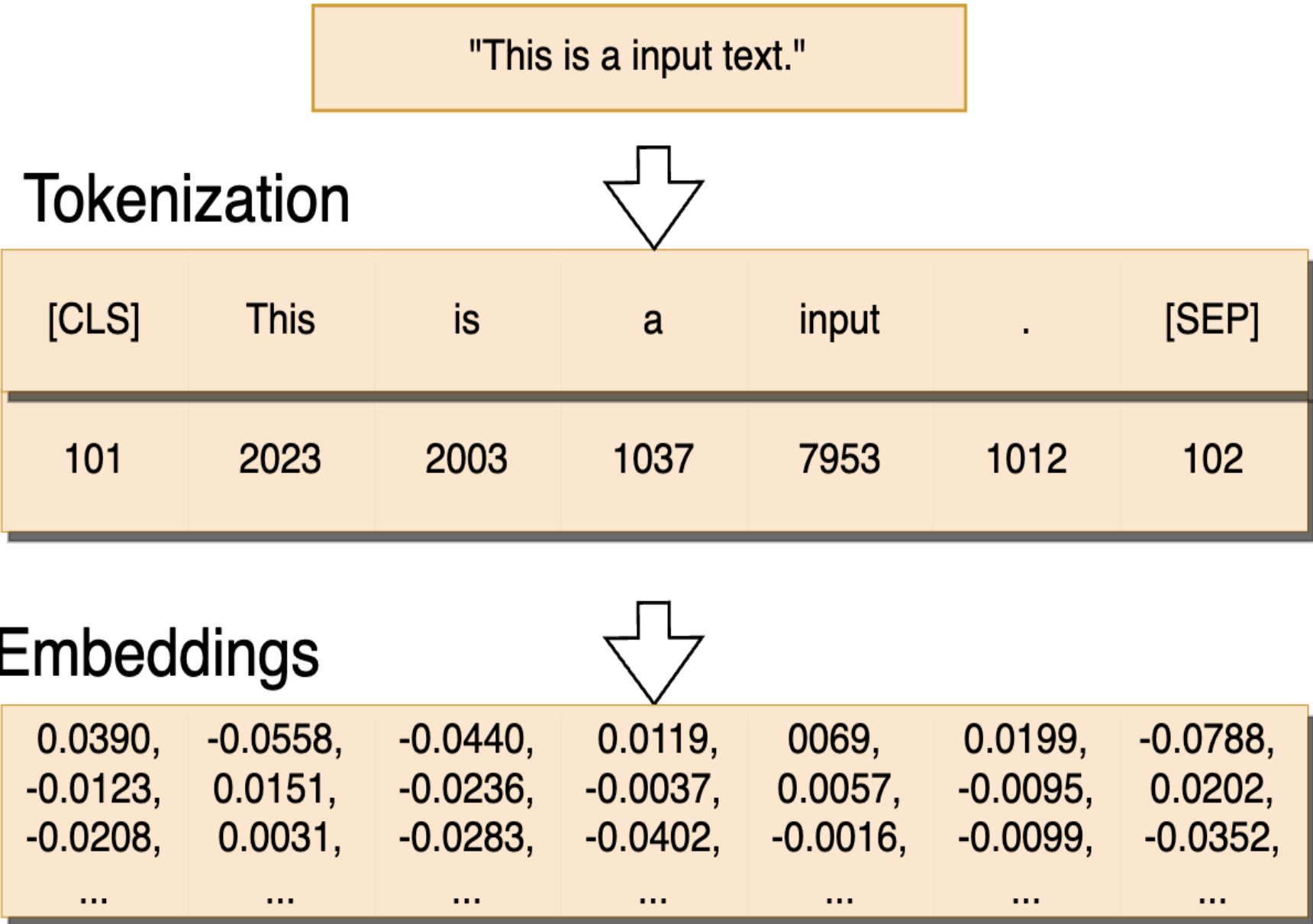
一、微调相关的基础概念

5. Embedding与位置编码

微调流程的数据流动路径



Tokenization 是“起点”——它决定了输入有多少个 token;
而 Embedding 和位置编码是“入口”——它们决定了这些 token 如何变成模型能处理的数学表示。



一、微调相关的基础概念

6. Embedding与位置编码：为什么微调要冻结部分层？

- Embedding层：单词的“数字身份证”
- Embedding 层的参数量有多大？
 - 在 BERT-base (110M 参数) 中，embedding 层约占 35% 的总参数
- 冻结 embedding 层：保持预训练的语义空间不变，只微调上层逻辑
- Word2Vec的局限性：
 - 每个词只有一个固定向量（“bank” 在 “river bank” 和 “bank account” 中一样）
 - 无法处理多义词
 - 必须提前训练，难以融入下游任务

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

- Positional Encoding: 让 Transformer “知道顺序”

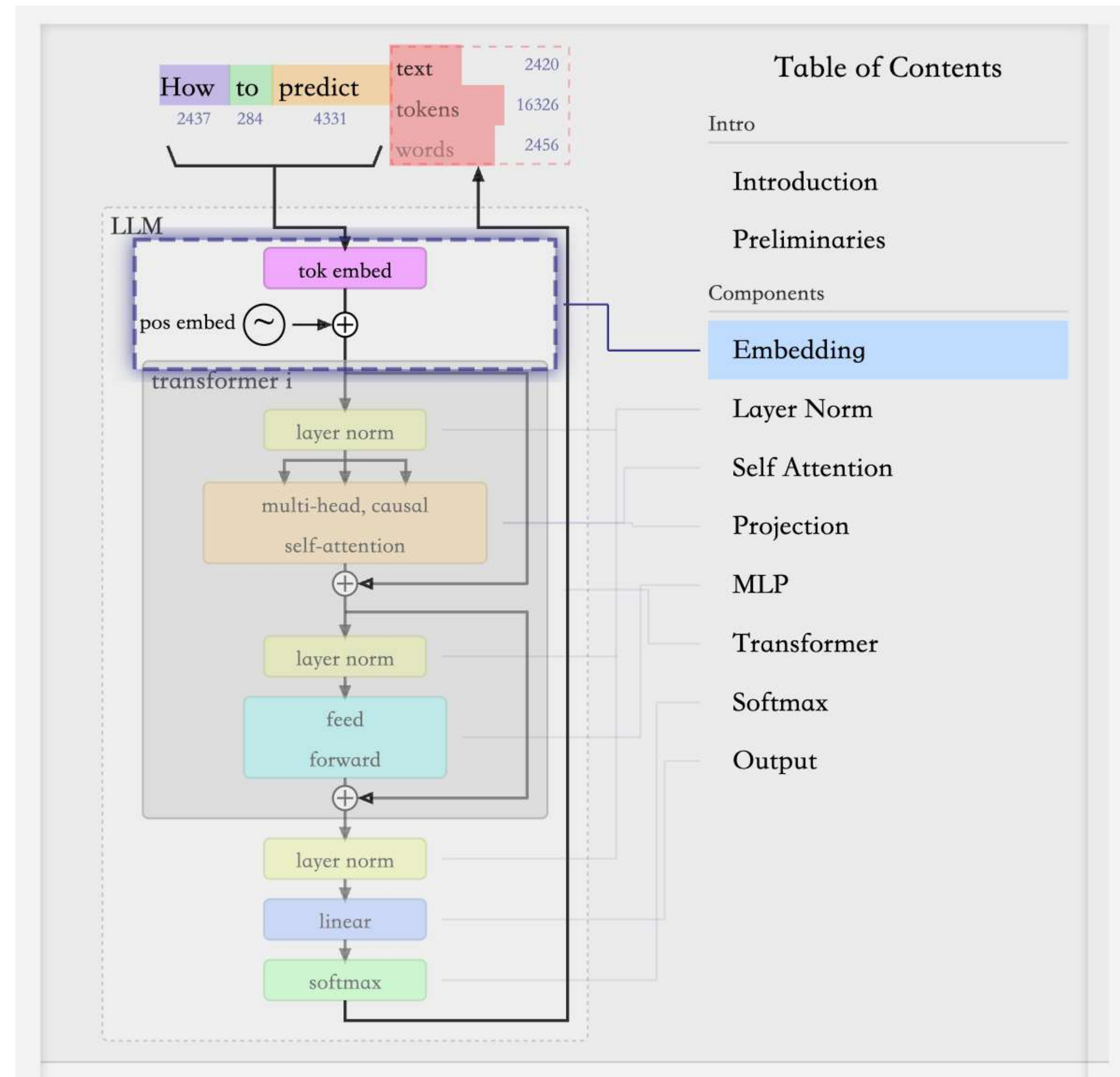
Transformer架构

Transformer由论文《Attention is All You Need》提出。

<https://arxiv.org/abs/1706.03762>

Transformer 由 Encoder 和 Decoder 两个部分组成，Encoder 和 Decoder 都包含 6 个 block。

<https://bbycroft.net/llm>



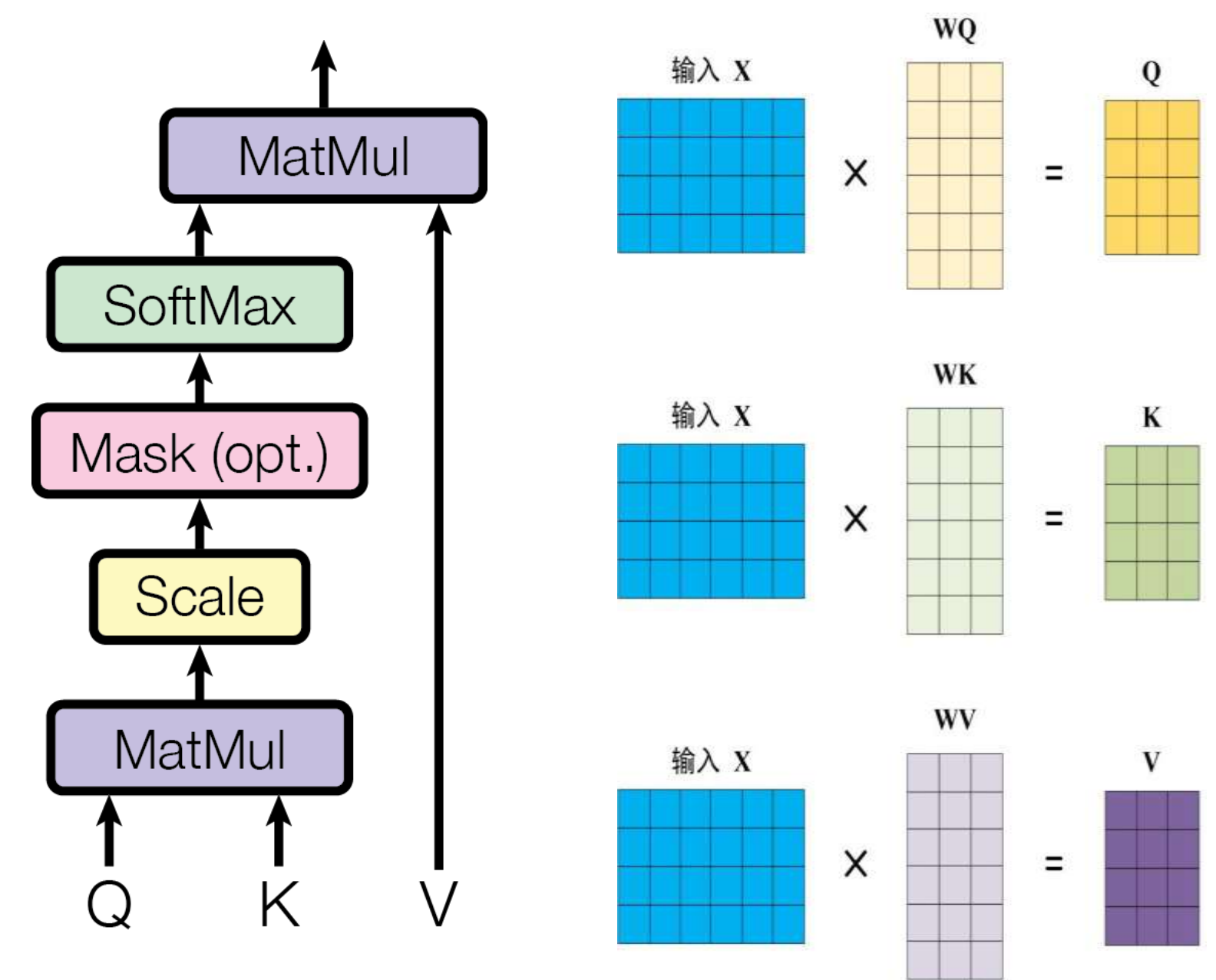
Self-Attention: 显存爆炸的根源

右图展示了 Self-Attention 的结构。该机制在计算过程中需要使用三个关键矩阵：Q（查询矩阵）、K（键矩阵）和 V（值矩阵）。在实际应用中，Self-Attention 的输入通常是词向量构成的矩阵 X（即各个单词的表示向量），或者是前一个编码器模块（Encoder block）的输出。Q、K、V 并非直接给定，而是通过对输入矩阵进行不同的线性变换（即分别乘以可学习的权重矩阵）得到的。

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

d_k 是 Q, K 矩阵的列数，即向量维度

在公式中，通过计算矩阵 Q 和 K 中各行向量之间的内积来衡量对应单词间的相关性。为了防止内积结果过大、导致后续 softmax 函数梯度消失或数值不稳定，将结果除以 K 向量维度的平方根。Q 与 K 的转置相乘后，得到一个大小为 $n \times n$ 的矩阵（其中 n 是句子中单词的数量），该矩阵的每个元素表示句子中第 i 个词与第 j 个词之间的注意力强度，反映了它们之间的关联程度。



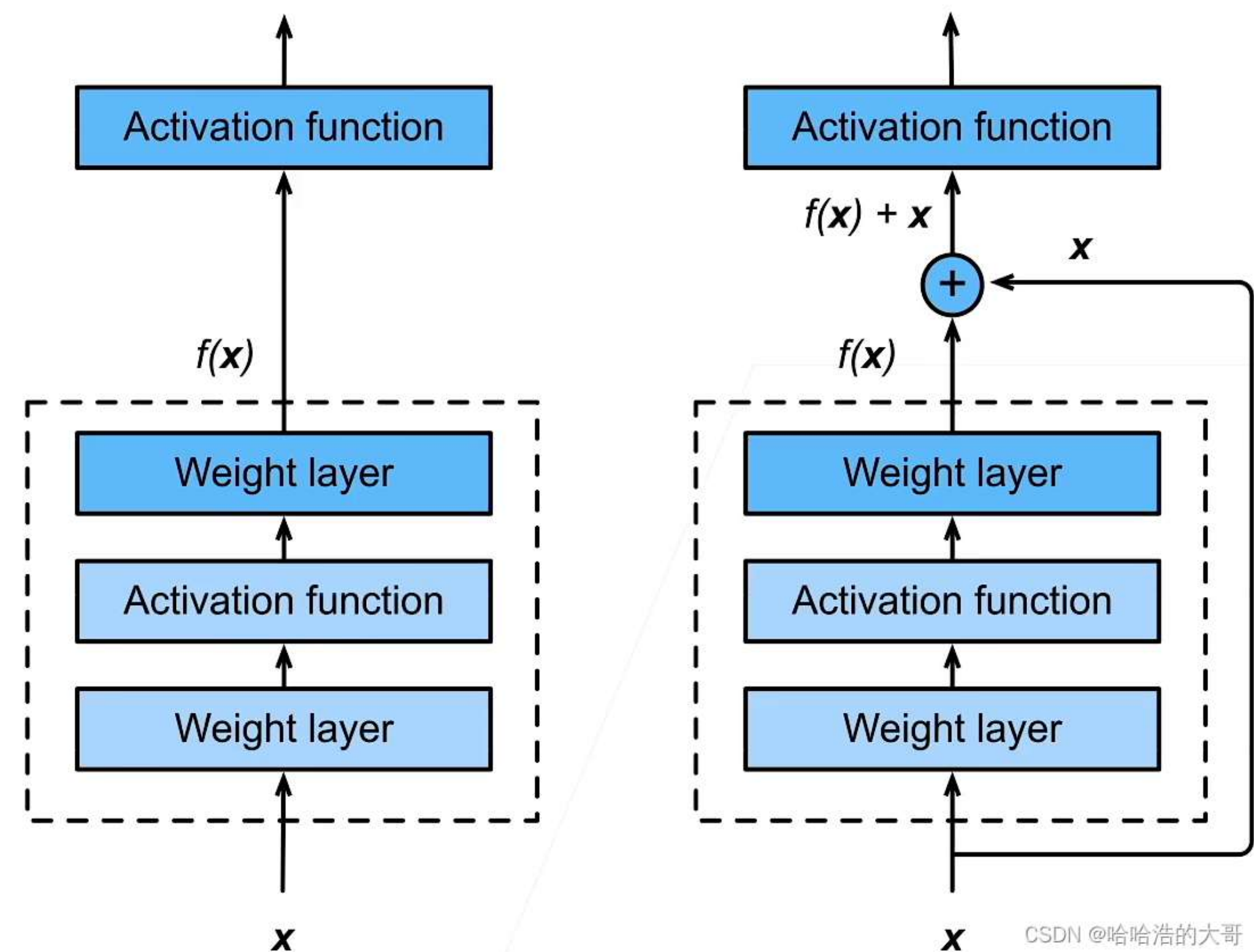
残差连接（Residual Connection） 和 层归一化（LayerNorm）

残差连接缓解梯度消失，使小数据微调成为可能

在每个 Transformer 子层（如 Self-Attention 或 FFN）之后，都有这样一条“高速公路”：

这条“+ Input”的路径，就是残差连接（Residual Connection）。

$$\frac{\partial}{\partial x}(x + F(x)) = 1 + \frac{\partial F}{\partial x}$$



CSDN @哈哈浩的大哥

Encoder-Decoder对比：为什么LoRA更适合Decoder-only模型

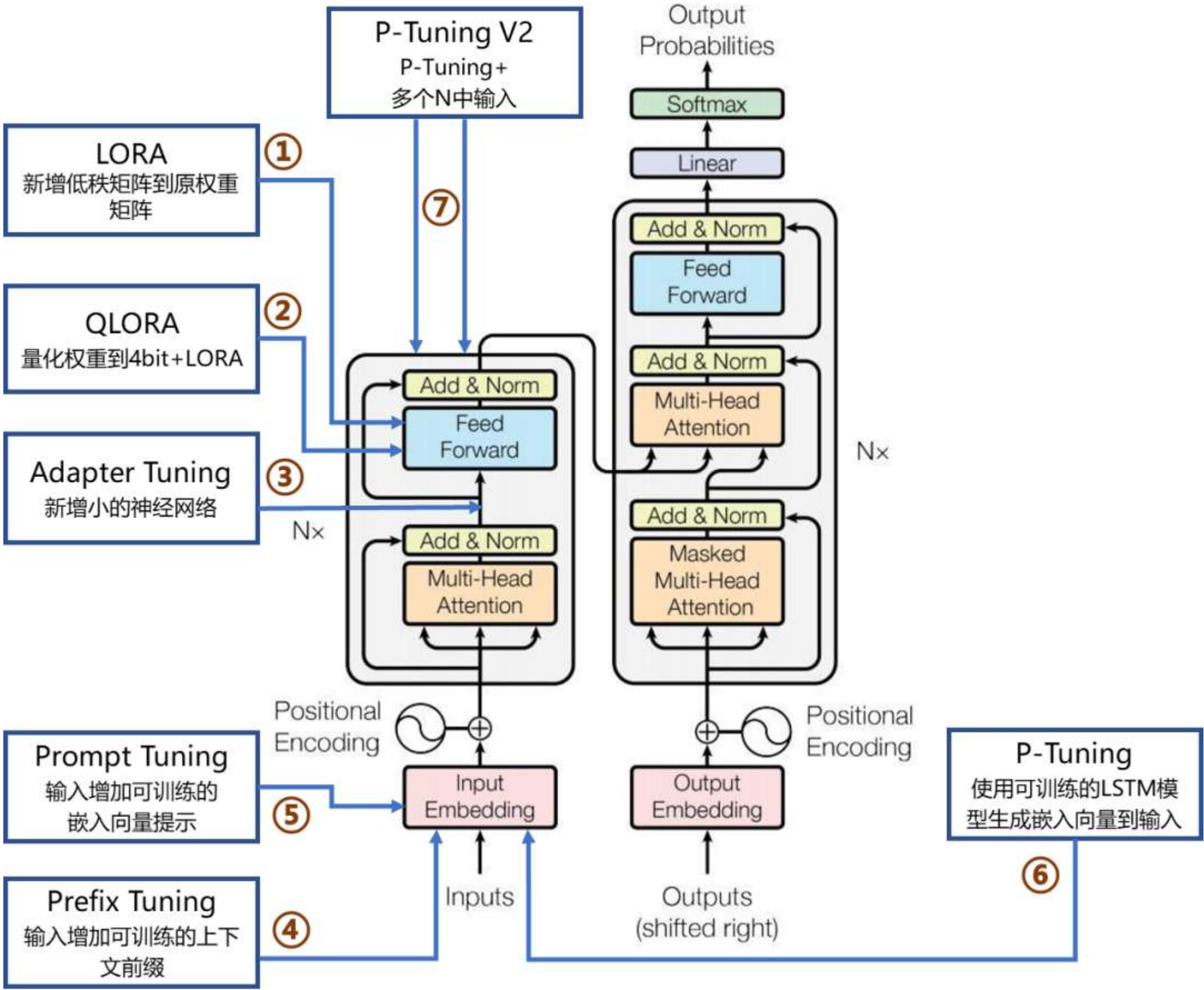
BERT（Encoder） vs GPT（Decoder） 架构差异

维度	Decoder-only（GPT）	Encoder-only（BERT）	Encoder-Decoder（T5/BART）
是否必须用 LoRA	是（模型太大）	否（任务简单，可全调）	视情况而定
LoRA 增益是否明显	显著（节省 90%+ 显存）	一般（原模型已高效）	明显但复杂
部署灵活性	高（单头生成）	低（需额外头）	中等
结构兼容性	极佳（仅需改 Q/V）	一般（可能干扰双向性）	复杂（双侧更新）

小结

技术细节	在Lora 中如何体现
Tokenization & 序列长度 L	LoRA 不能减少 attention matrix 的L2开销，但它减少了参数更新量，间接缓解长序列微调压力
d_model（即 d）	LoRA 的核心低秩矩阵就是基于d设计的，比如 $W=W_0+\Delta W=W_0+B\cdot A$ ，其中 $B\in R^{d\times r}$, $A\in R^{r\times d}$
Self-Attention 中的 QKV 矩阵	LoRA主要就加在 Q、K、V 的投影层上，因为这些层参数多、更新代价高
显存爆炸问题	LoRA 正是为了绕过“全量微调所有参数”带来的显存和计算压力而提出的

三、高效微调：从理论到LoRA的必然选择



方法	显存消耗	推理延迟	适配灵活性	适用场景
全量微调	极高（更新全部参数）	无	✗ 差（每个任务存一份完整模型）	数据量极大、预算充足
Adapter	中（插入小型FFN层）	有（增加网络层数）	好（可插拔）	多任务切换、中间层适配
Prefix Tuning	低（仅优化前缀向量）	有（输入变长）	好	生成任务、轻量控制
Prompt Tuning	低（软提示向量）	无	好	大规模统一接口
LoRA	极低（仅更新低秩矩阵）	无（合并后等价原模型）	极好（可热插拔）	90% 业务场景

三、高效微调：从理论到LoRA的必然选择

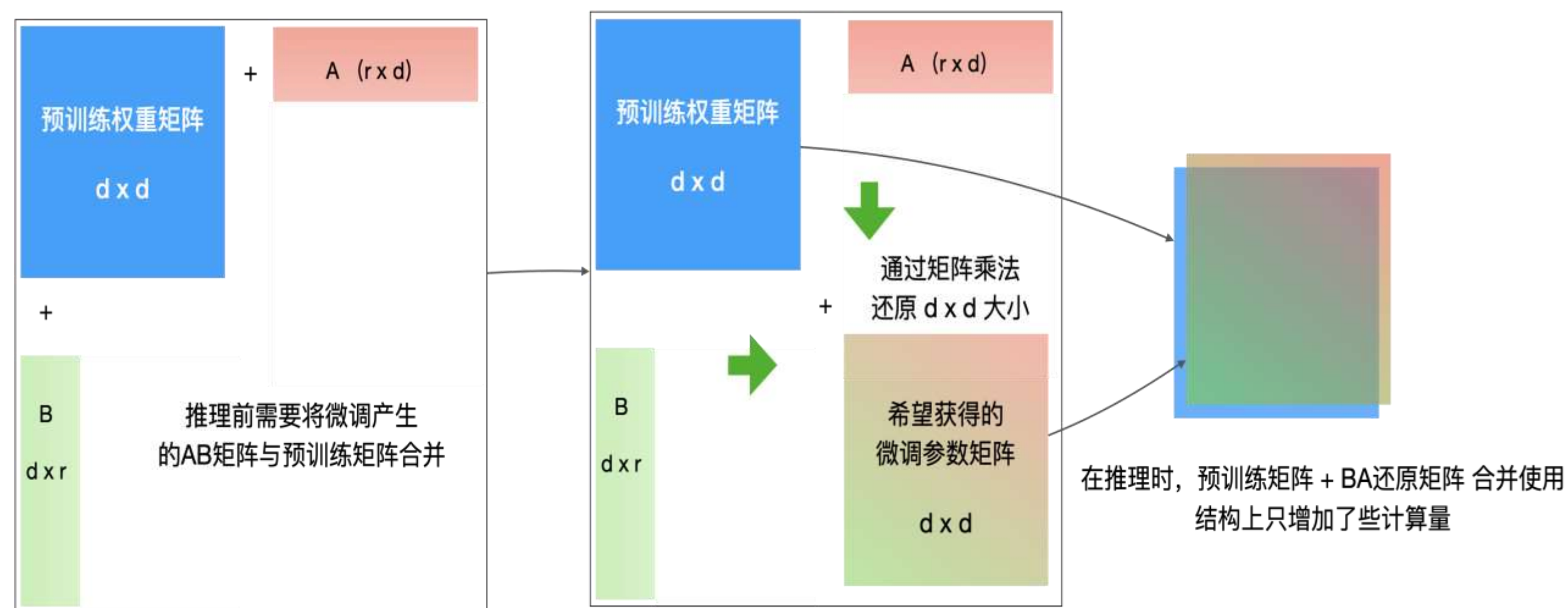
LoRA 的核心思想是：

“不直接更新大矩阵 $W \in \mathbb{R}^{d \times k}$ ，而是用两个小矩阵的乘积来近似它的变化： ”

$$W' = W + \Delta W = W + B \cdot A$$

其中：

- $B \in \mathbb{R}^{d \times r}$
- $A \in \mathbb{R}^{r \times k}$
- $r \ll d, k$ （例如 $r = 8$ 或 64 ，而 $d = 768$ ）



四、工程实战： 用LoRA解决真实业务问题

2、 意图识别

https://bailian.console.aliyun.com/?tab=model#/model-market

1 数据管理

2 模型调优

3 模型部署

超参配置 恢复默认

循环次数 ?

3

学习率 ?

1e-4

批次大小 ?

16

更多配置 ^

学习率调整策略 ?

linear

验证步数 ?

50

序列长度 ?

8192

学习率预热比例 ?

0.05

权重衰减 ?

0.01

LoRA秩值 ?

8

LoRA阿尔法 ?

16

LoRA丢弃率 ?

0.1

LoRA目标模块 ?

ALL

```
1 2025-08-17 16:31:25,527 - INFO - data process succeeded, start to fine-tune
2 Fine-tune started
3 dataset info: {'train_dataset': {'total': 2504, 'mean': 33.386666666666666, 'std': 4.295403227741127, 'min': 28, 'max': 45, 'size': 75}, 'val_dataset': {'total': 332, 'mean': 36.888888888888886, 'std': 4.254264382017033, 'min': 29, 'max': 42, 'size': 9}, 'final_dataset': {'total': 2504, 'mean': 33.386666666666666, 'std': 4.295403227741127, 'min': 28, 'max': 45, 'size': 75}, 'total_token': 2504}
4 {'loss': 6.707875728607178, 'learning_rate': 0.0001, 'train_speed(iter/s)': 0.149563, 'epoch': 0.222222222222222, 'consumed_train_tokens': 525}
5 {'loss': 6.232982158660889, 'learning_rate': 6.363636363636364e-05, 'train_speed(iter/s)': 0.413845, 'epoch': 1.0, 'consumed_train_tokens': 2408}
6 {'loss': 4.317822265625, 'learning_rate': 1.8181818181818182e-05, 'train_speed(iter/s)': 0.527731, 'epoch': 2.0, 'consumed_train_tokens': 4816}
7 {'eval_loss': 3.39978289604187, 'eval_runtime': 0.3084, 'eval_samples_per_second': 29.185, 'eval_steps_per_second': 9.728, 'epoch': 2.4444444444444446}
8 {'train_runtime': 19.4867, 'train_samples_per_second': 11.546, 'train_steps_per_second': 0.616, 'train_loss': 5.004913171132405, 'epoch': 2.4444444444444446, 'consumed_train_tokens': 5879}
9 Actual number of consumed tokens is 5879!
10 Fine-tune succeeded!
11 2025-08-17 16:34:36,935 - INFO - fine-tuned output got, start to transfer it for inference
12 2025-08-17 16:35:22,243 - INFO - transfer for inference succeeded, start to deliver it for inference
13 2025-08-17 16:36:51,930 - INFO - start to save checkpoint
14 2025-08-17 16:38:04,999 - INFO - training usage 5879
15 2025-08-17 16:38:04,532 - INFO - finetune-job succeeded
16 2025-08-17 16:38:05,013 - INFO - ##FT_COMPLETE##
```

四、工程实战：用LoRA解决真实业务问题

2、意图识别

<https://bailian.console.aliyun.com/?tab=model#/model-market>

4 部署计费

5 文本模型

计费方式

根据业务场景，选择合适的计费方式，详情请见：[计费说明](#)



Tokens调用计费/后付费



根据模型调用产生的Tokens量计费，适用于峰谷调用、效果验证场景，超过1个月无调用量，模型服务将自动下线

灵活使用，按用量计费

RPM ⓘ 180

TPM ⓘ 1500000

部署时长 超过1个月无调用量，模型服务将自动下线

[计费详情](#)

[价格说明](#)

Lora 优势总结

全量微调	LoRA 微调
全量微调参数太多，显存不够	只更新少量低秩矩阵，显存下降 70%~90%
Self-Attention 层参数量大 (d^2)	在 QKV 层用 LoRA，避免更新大矩阵
长序列导致激活值显存爆炸	虽不能减少 attention matrix，但减少了 optimizer states 和 gradient 存储
微调成本高，难以部署多个任务	一个基础模型 + 多个 LoRA 适配器，轻松切换任务

目录

阶段	主题	核心工程理念
Phase 0	工程准备：AI开发环境与项目规范	环境可复现、代码可管理
Phase 1	数据工程：NLP数据处理与特征管理	数据版本化、可追溯
Phase 2	模型开发：Transformer与高效微调	模块化、参数高效、可复现
Phase 3	模型训练与实验管理	实验可追踪、超参可优化
Phase 4	模型评估与上线决策	可测性、A/B测试、因果推断
Phase 5	模型压缩与部署	轻量化、边缘部署、服务化

03 实验管理与超参数优化

一、实验管理

1.1、什么是实验管理？

实验管理 = 用最少实验次数，证明模型能提升核心业务指标

1.2、实验管理对大模型微调的价值是什么？

- 快速定位最优配置
- 避免线上灾难
- 说服老板的关键证据

1.3、怎么做实验管理？关键检查点

- 实验名是否回答具体决策问题？（例：lora_alpha_2x_vs_4x）
- 是否包含bad case解决率？（业务安全底线）
- 是否只保存合并后的模型？

二、超参优化：实战 LoRA调参 实验管理

2.1 微调时需要关注哪些参数需优化

0. $f(x) = ax$

1. Loss Function & Cost Function - 量化评估模型表现

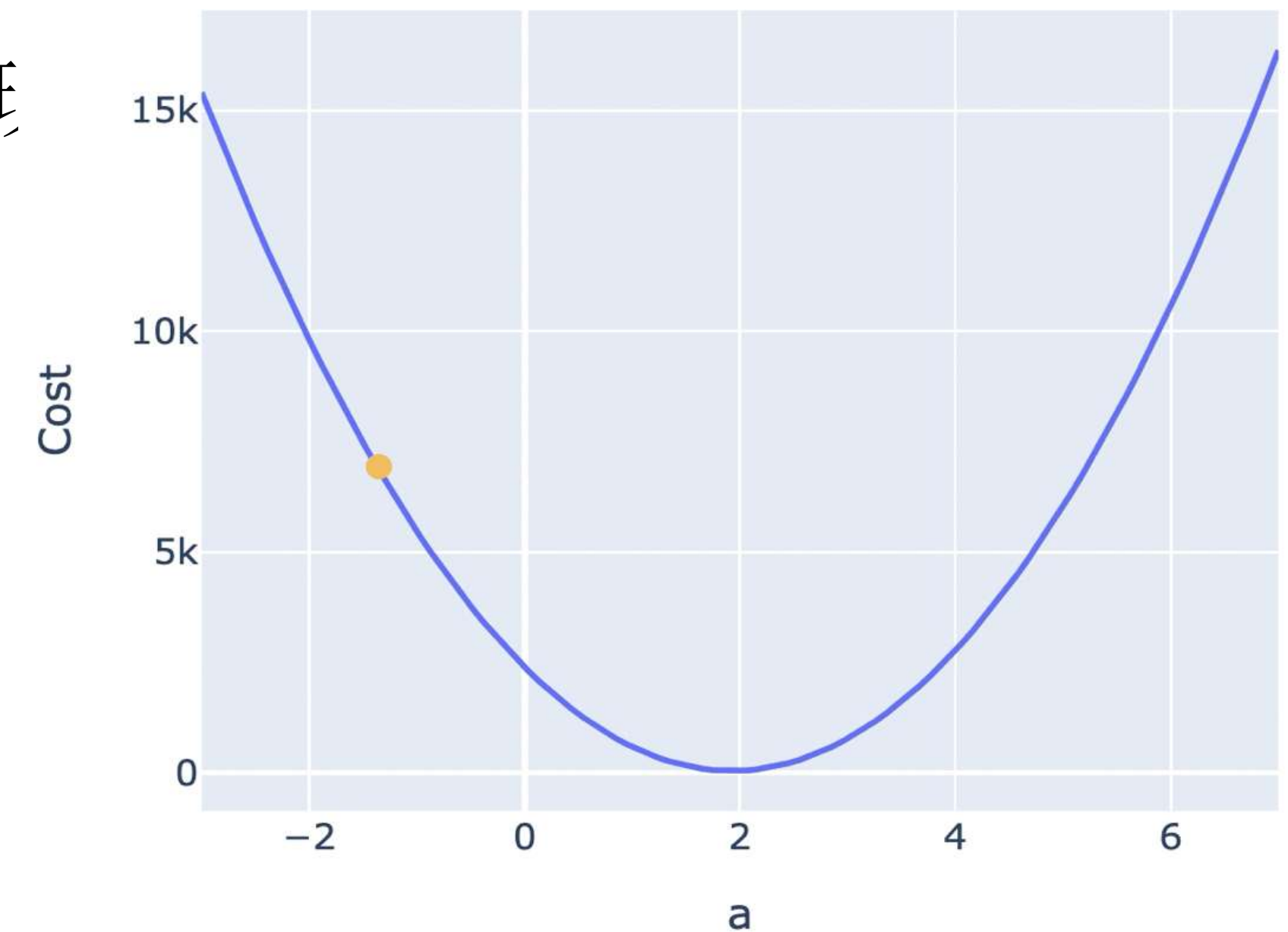
2. 调整参数的幅度（学习率 Learning Rate）

3. batch size

4. eval steps

5. epoch

6. 三套题目： 训练集 、测试集、验证集



三、一次典型的微调实验

1. 基准模型测试

2. 迭代式调参实验

第一次实验：学习率过高 → 训练失败（loss 上升）

第二次实验：大幅降低学习率 → 欠拟合（loss 持续下降）

第三次实验：增加 epoch → 过拟合（验证 loss 上升）

第四次实验：扩充数据至 1k 条，提升 lora_rank=8 → 欠拟合

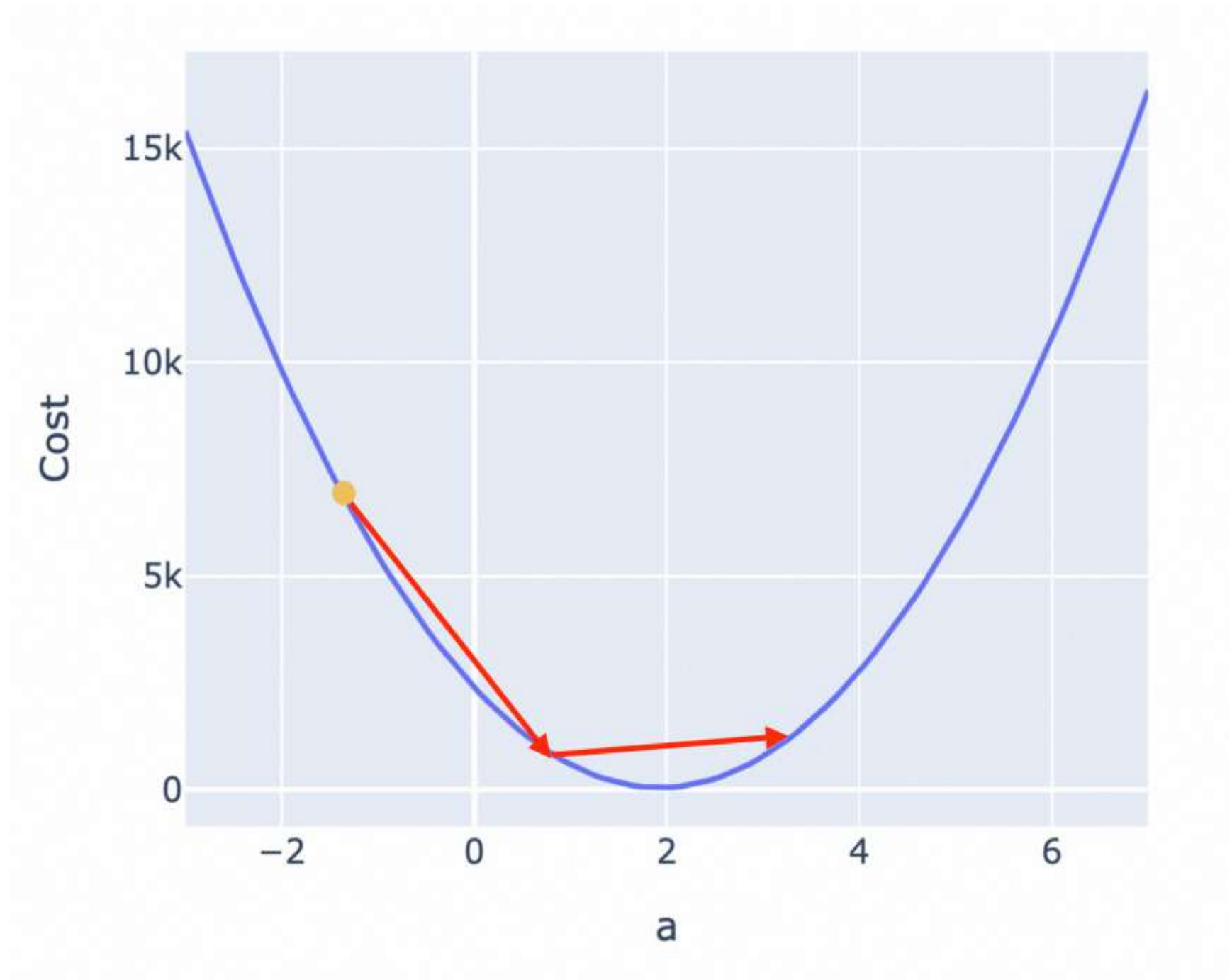
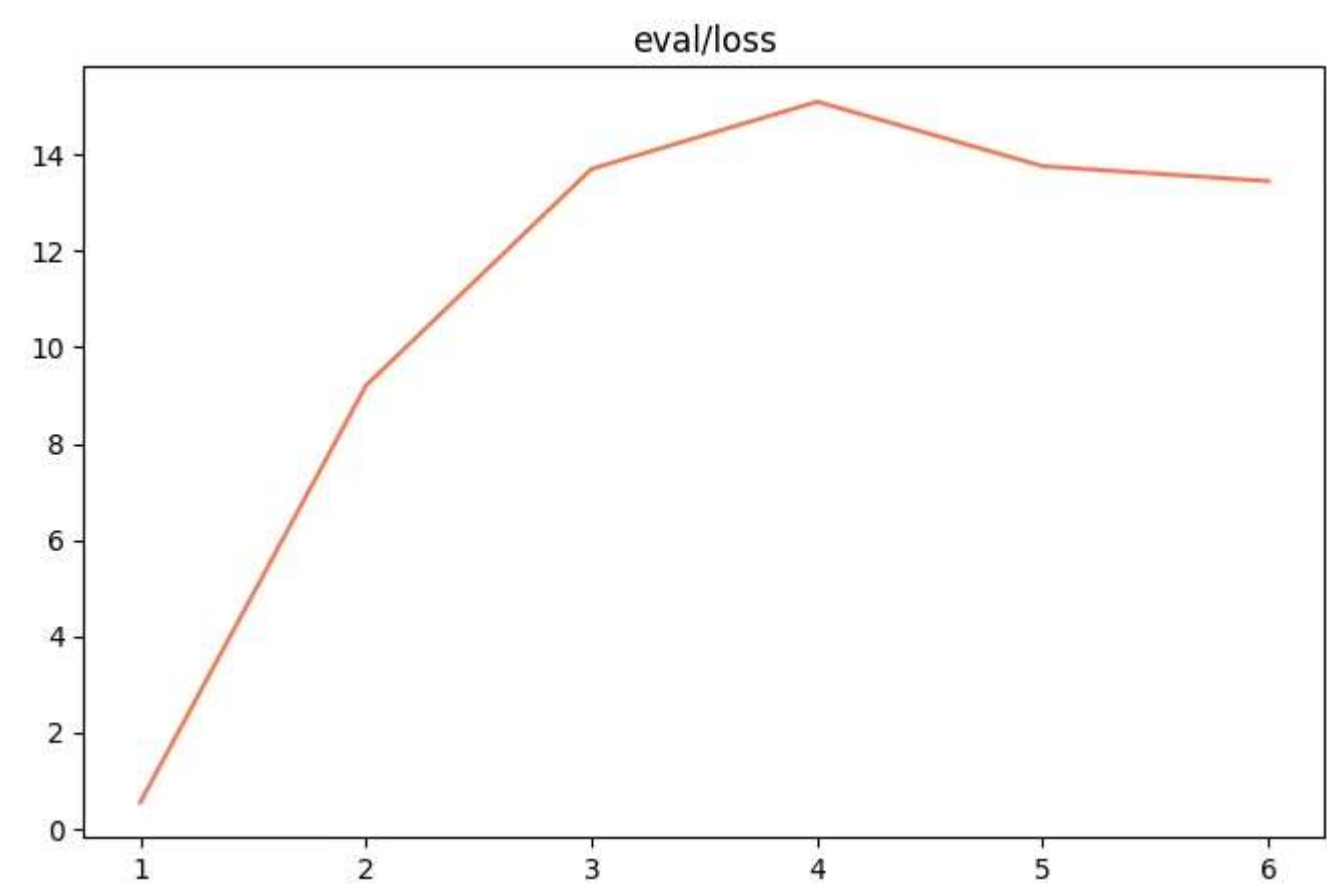
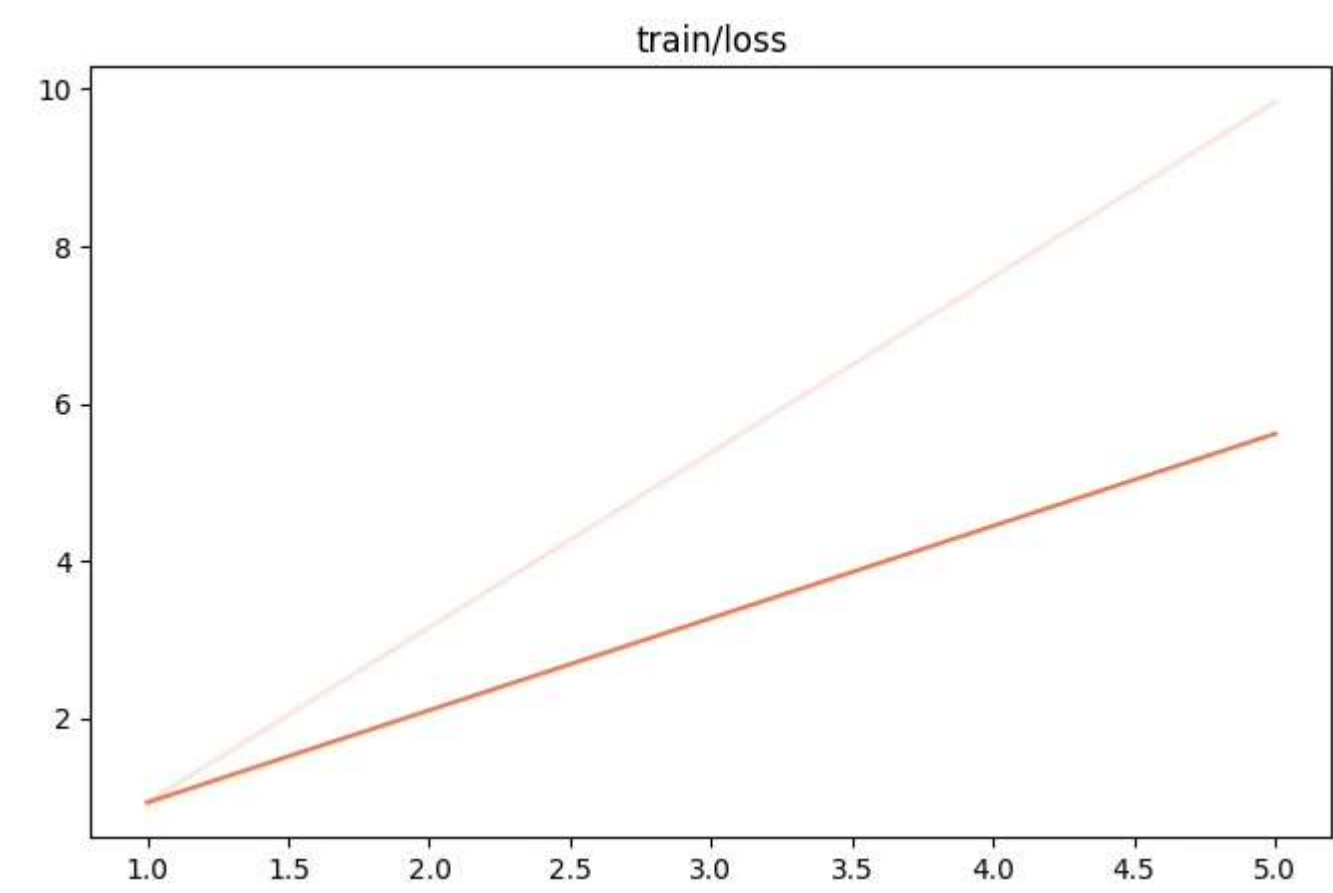
第五次实验：增加 epoch 至 15 → 训练成功（loss 收敛）

3. 微调后评估

4. 参数矩阵融合

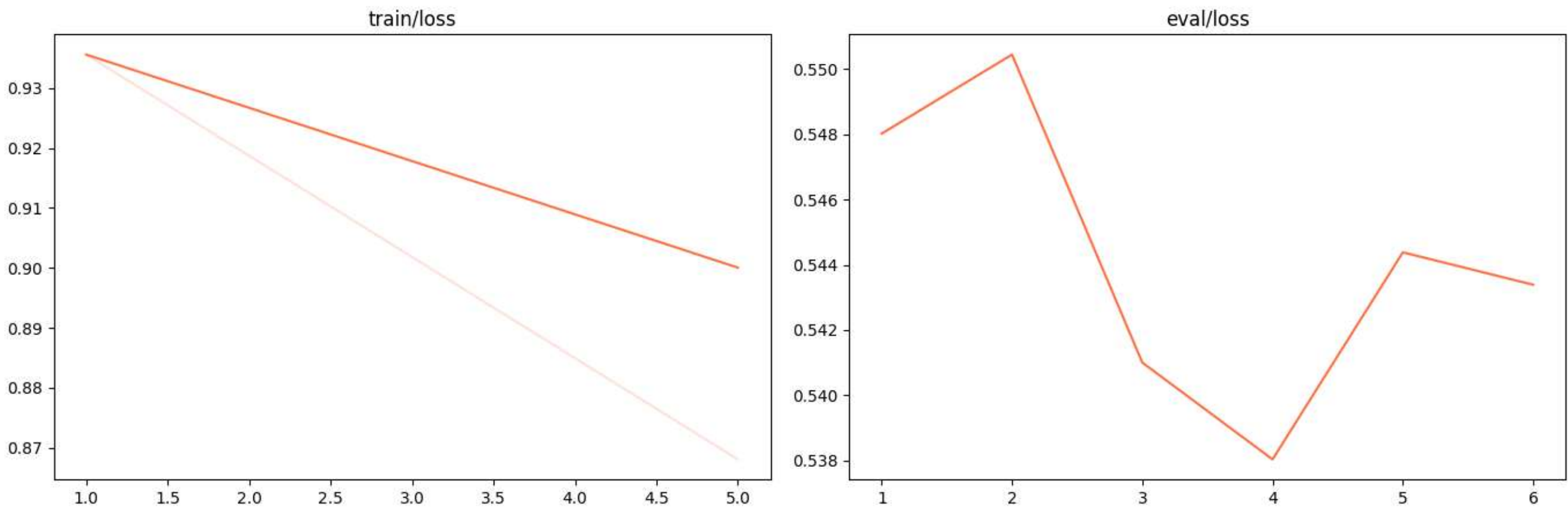
第一次实验

参数	参数值
学习率 (learning_rate)	0.1
LoRA的秩 (lora_rank)	4
数据集学习次数 (num_train_epochs)	1



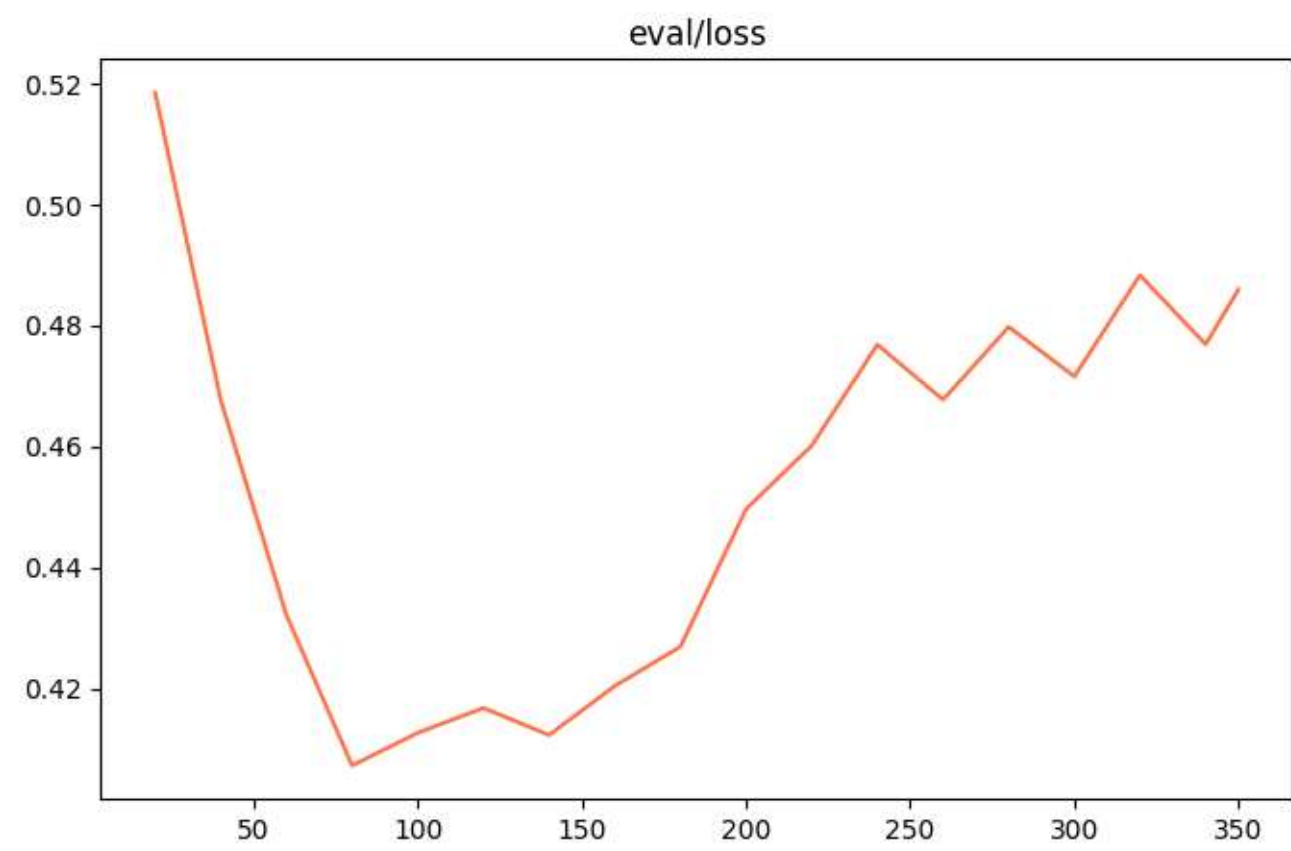
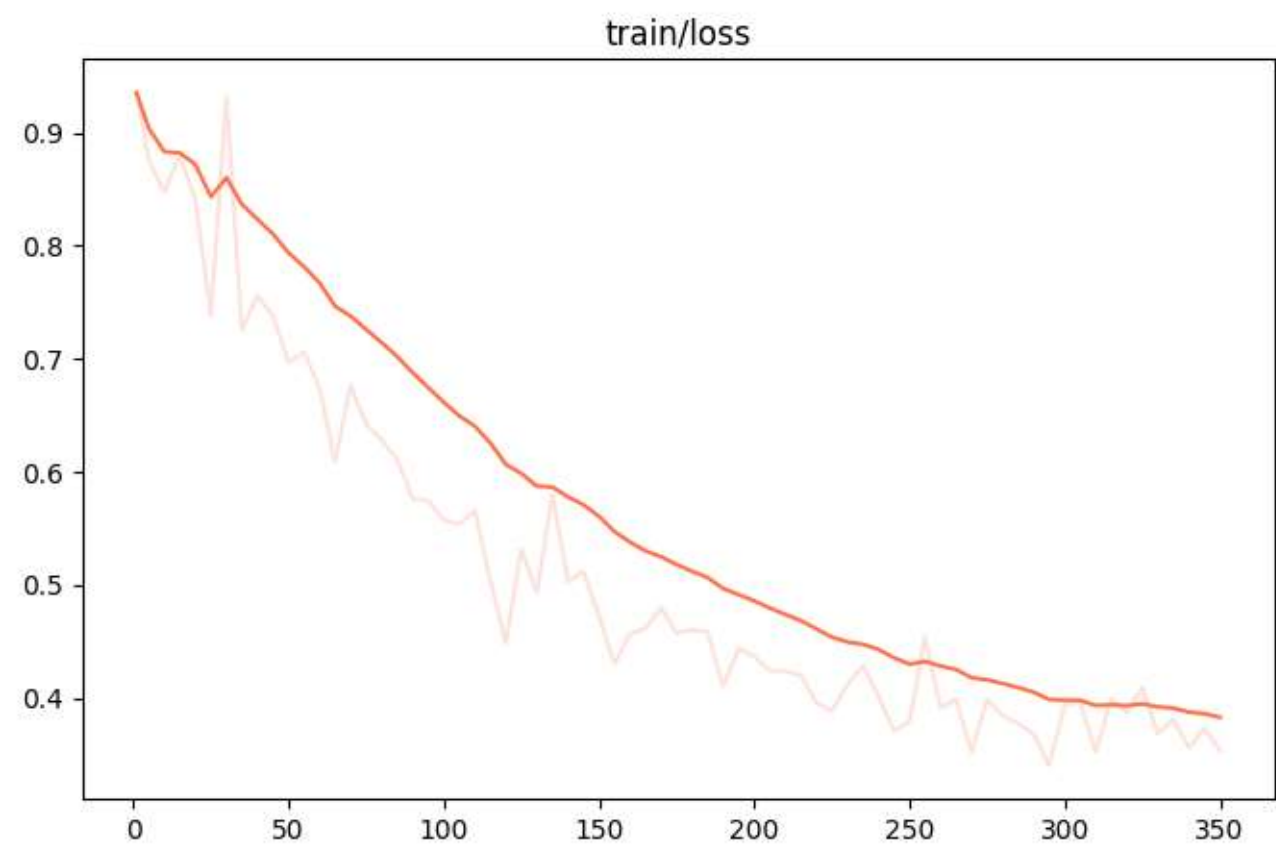
第二次实验

参数	参数值
学习率 (learning_rate)	0.00005
LoRA的秩 (lora_rank)	4
数据集学习次数 (num_train_epochs)	1



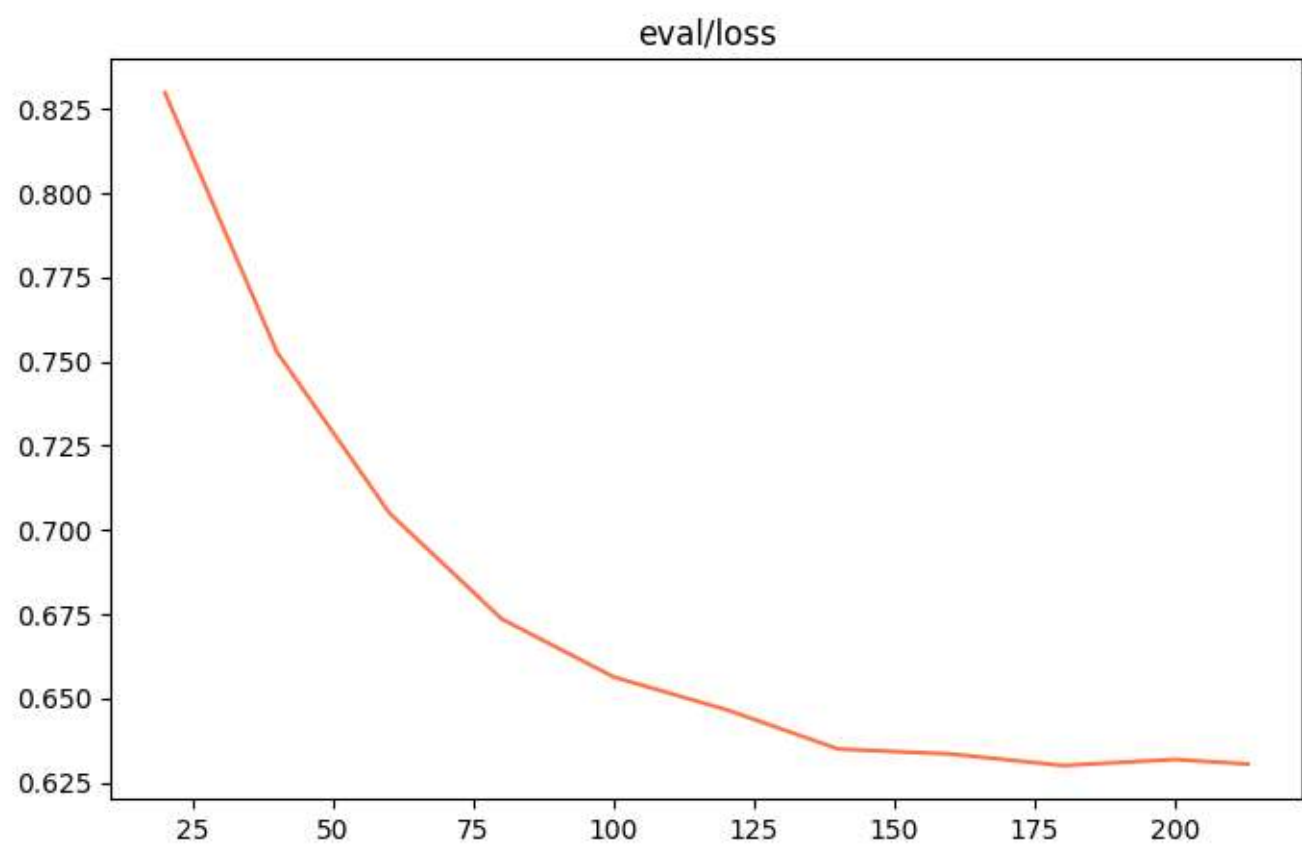
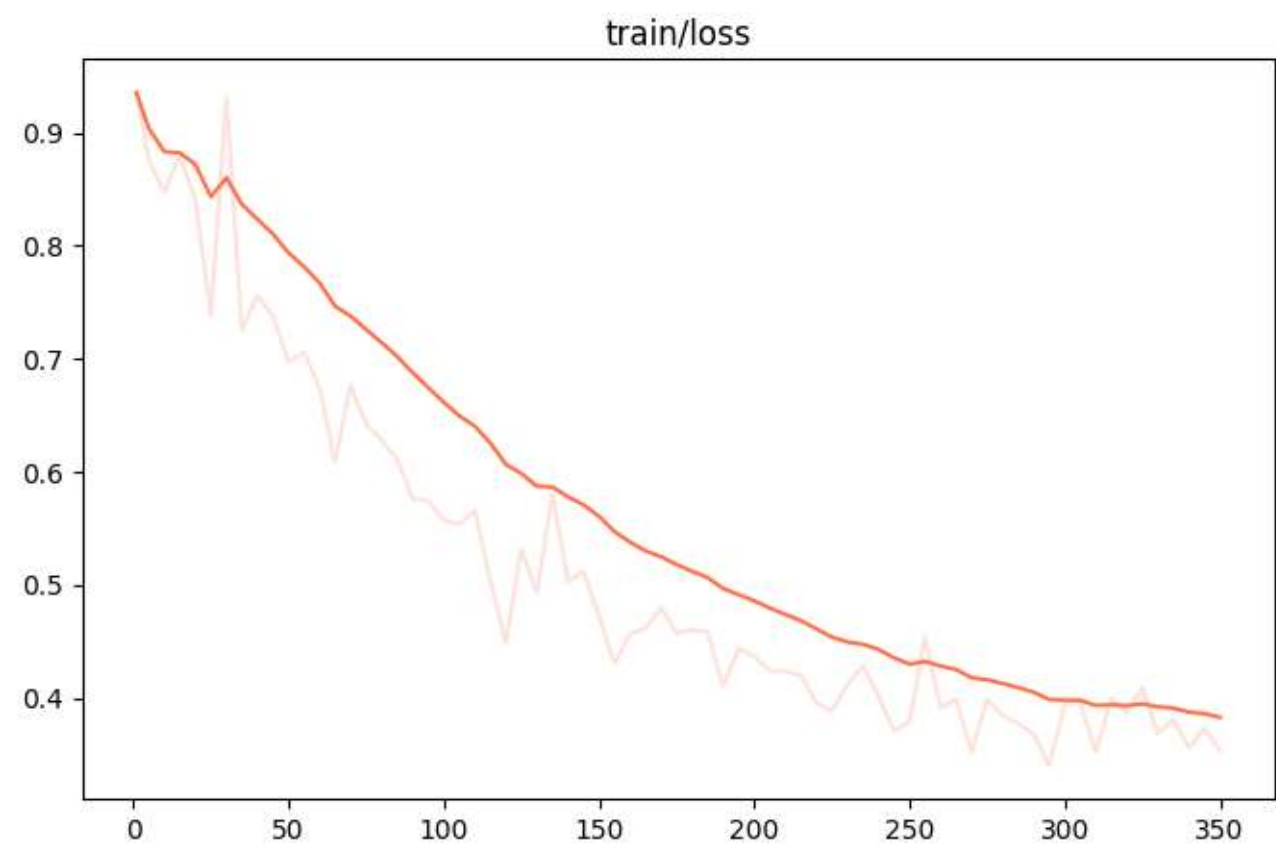
第三次实验

参数	参数值
学习率 (learning_rate)	0.00005
LoRA的秩 (lora_rank)	4
数据集学习次数 (num_train_epochs)	50



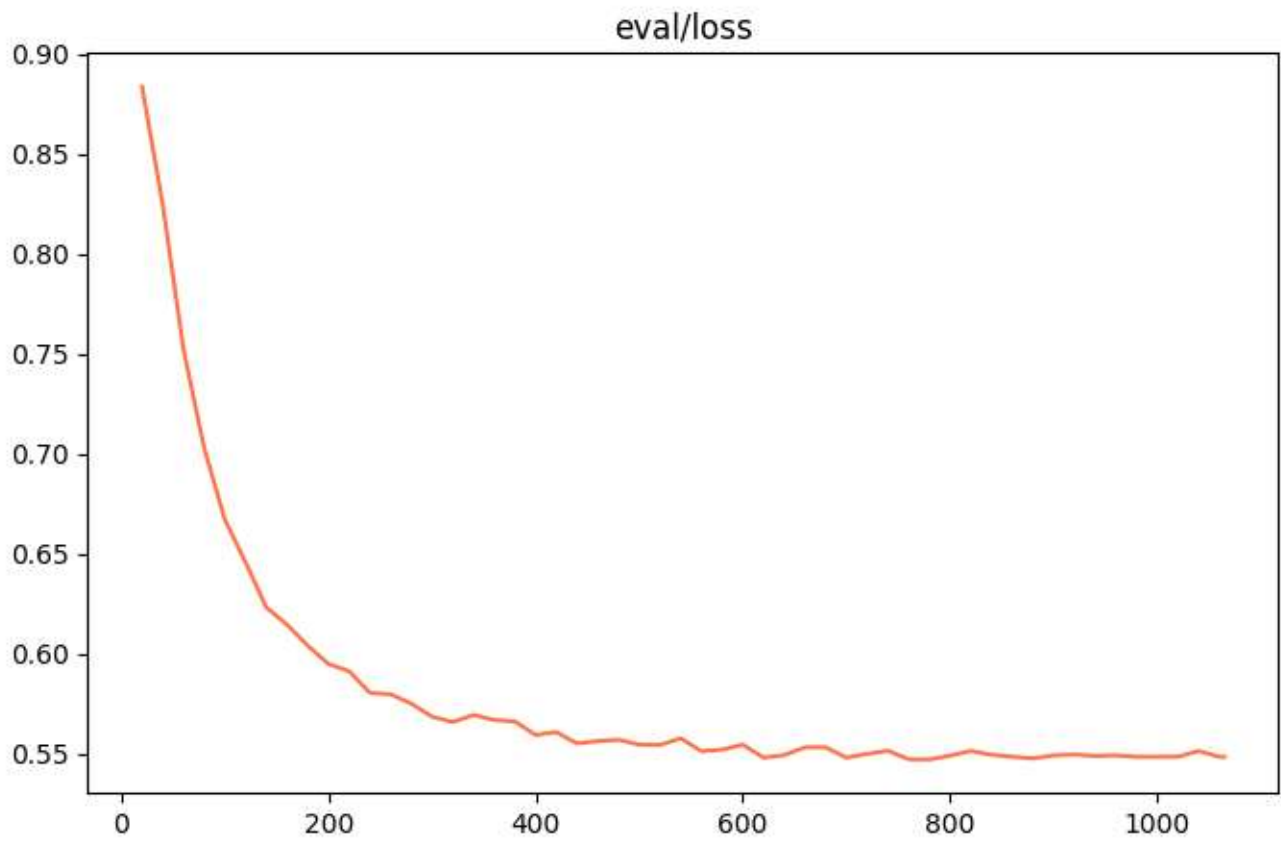
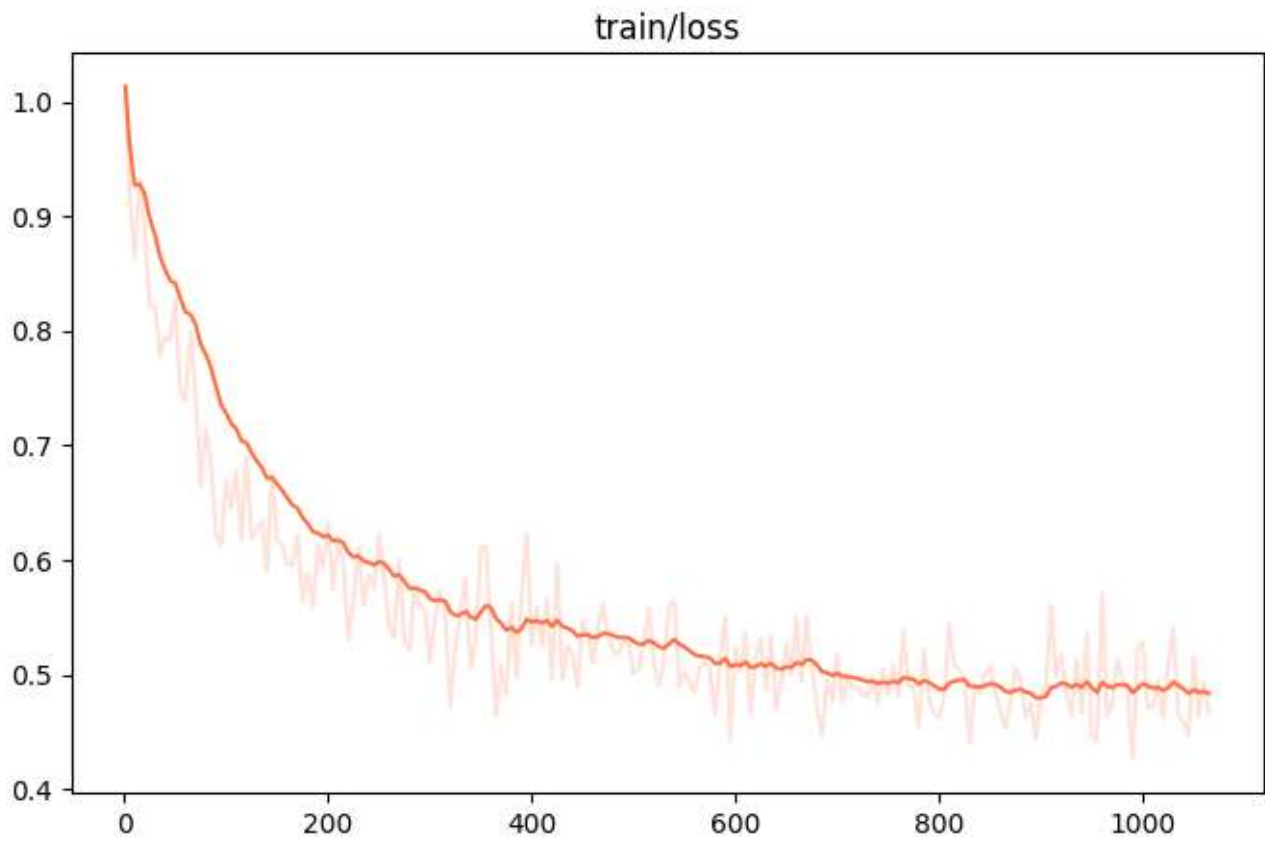
第四次实验

参数	参数值
学习率 (learning_rate)	0.00005
LoRA的秩 (lora_rank)	8
数据集学习次数 (num_train_epochs)	3
增加数据集	1000+条以上



第五次实验

参数	参数值
学习率 (learning_rate)	0.00005
LoRA的秩 (lora_rank)	8
数据集学习次数 (num_train_epochs)	15
增加数据集	1000+条以上



测试模型前后变化

微调后一般会保存两个checkpoint文件：

`best_model_checkpoint`（在验证集表现最佳的模型参数）

`last_model_checkpoint`（微调任务完成时的模型参数）。

一般选取 `best_model_checkpoint` 地址替换下面代码中的`ckpt_dir`

参数矩阵融合

```
swift export \  
--ckpt_dir 'output/qwen2_5-1_5b-instruct/vx-xxx/checkpoint-xxx\  
--merge_lora true
```

融合后的完整参数矩阵默认存储在checkpoint目录下

小结

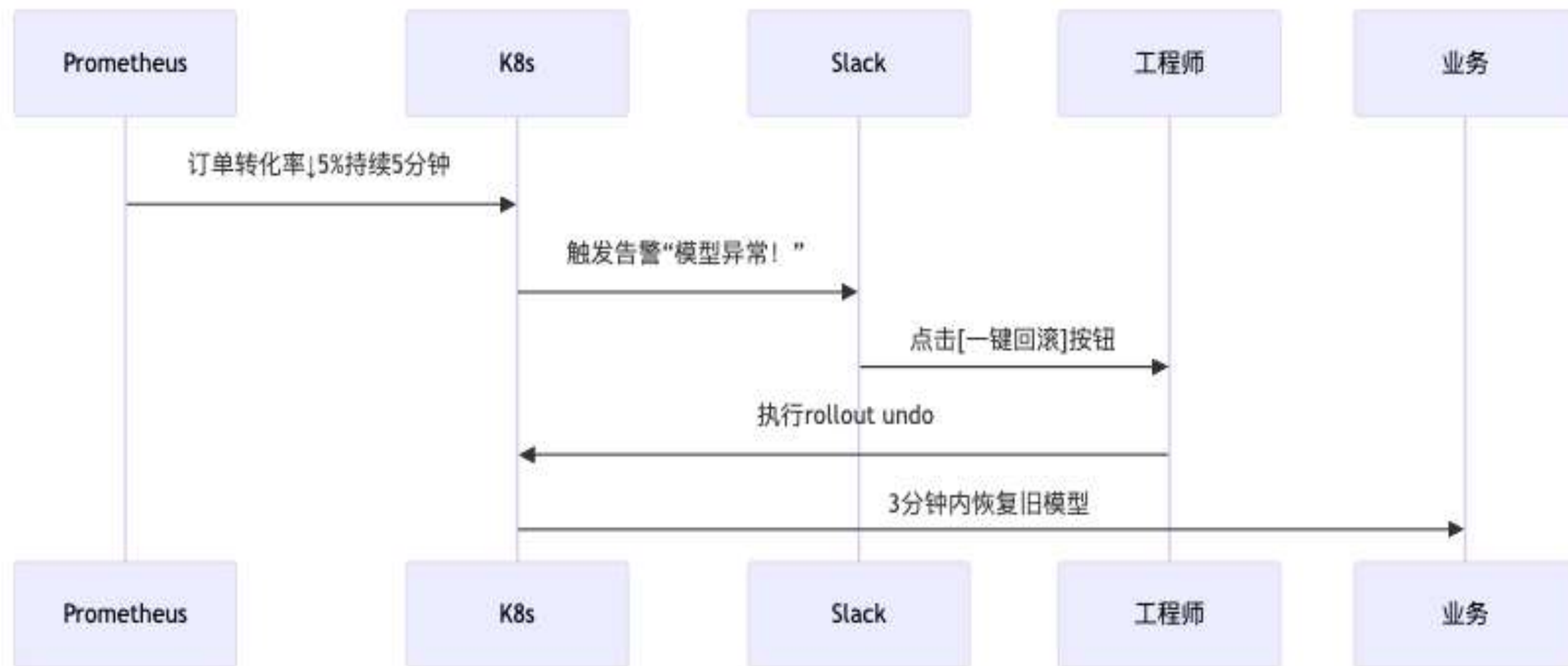
- 微调是提升模型特定任务能力的有效手段，尤其适用于提示工程和 RAG 无法解决的复杂推理任务。
- 关键训练参数包括：学习率、batch size、epoch、lora_rank、eval_steps 等，需通过实验不断调优。
- 使用验证集 loss 监控训练状态，识别欠拟合与过拟合，指导调参方向。
- LoRA 是当前主流的高效微调方法，显著降低资源消耗，适合资源受限场景。

04 模型评估与上线决策

快速回滚SOP

- 关键证据链

- Bad Case解决率
- 订单转化率
- 回滚能力



- 业务最关心的三个问题

- 1. 上线后用户会更满意吗? → “Bad Case解决率+24%，投诉率预计↓30%”
- 2. 万一出问题怎么办? → “5分钟自动回滚，历史故障损失从\$200k→\$2k”
- 3. 值得花时间上线吗? → “每日GMV+¥15k，投入1人天，ROI=5000%”

05 模型压缩与部署

让大模型在有限的显存里跑起来

目标：用最简单的方式，把一个LoRA微调后的Qwen-7B意图识别模型，压缩到1.8GB以内，P99延迟82ms，支持5秒热更新

问题：

1. 为什么要压缩模型？有几种瘦身方式
2. 量化是怎么回事？
3. 怎么量化？怎么评估？

一、我们为什么要压缩模型？——从“跑不动”说起

常见场景：

你训练好的Qwen-7B模型，在本地能跑，但一上线就OOM

推理延迟300ms，用户体验差

解决方案：

模型压缩 —— 就像“给视频转成低码率MP4”一样，让模型变小、变快

二、模型压缩有哪几种“瘦身”方式？

压缩方式	场景
1. 量化 (Quantization)	显存↓6~8倍，推理快2~3倍，LoRA模型首选
2. 剪枝 (Pruning)	容易破坏模型逻辑，bad case暴增
3. 知识蒸馏 (Distillation)	需要训练，周期长，LoRA场景不适用

三、量化到底是啥？

量化过程：

把模型中“32位浮点数”换成“8位整数”

例如：0.78 $\rightarrow$ 200（通过缩放比例映射）

损失：

精度轻微下降（像GIF图有“色块”）

但对大多数任务（如意图识别）影响极小

四、怎么对 Qwen-7B 做 INT8 量化

第一步：先合并LoRA权重

```
CUDA_VISIBLE_DEVICES=0 \  
swift infer \  
    --adapters output/v0-20250820-011426/checkpoint-94/ \  
    --stream true \  
    --merge_lora true
```

```
root@dsw-1291123-84567c45f5-sjdqw:/mnt/workspace/demo1/output# cd v0-20250820-011426/  
root@dsw-1291123-84567c45f5-sjdqw:/mnt/workspace/demo1/output/v0-20250820-011426# ls  
args.json checkpoint-50 checkpoint-94 checkpoint-94-merged images logging.jsonl runs  
root@dsw-1291123-84567c45f5-sjdqw:/mnt/workspace/demo1/output/v0-20250820-011426# du -sh checkpoint-94*  
232M checkpoint-94  
15G checkpoint-94-merged  
root@dsw-1291123-84567c45f5-sjdqw:/mnt/workspace/demo1/output/v0-20250820-011426#
```


四、怎么对 Qwen-7B 做 INT8 量化

第二步：用 swift 做INT8量化

```
1  CUDA_VISIBLE_DEVICES=0 swift export \  
2      --model Qwen/Qwen2.5-7B-Instruct \  
3      --quant_bits 8 --quant_method bnb \  
4      --dataset AI-ModelScope/alpaca-gpt4-data-zh \  
5      --output_dir Qwen2.5-7B-Instruct-BNB
```

第三步：推理

```
1  CUDA_VISIBLE_DEVICES=0 \  
2  swift infer \  
3      --model ./Qwen2.5-7B-Instruct-BNB/ \  
4      --stream true \  
5      --temperature 0 \  
6      --max_new_tokens 2048  
7
```

五、Ollama 能用来推理吗？

功能丰富， 建议开发环境

Parallel request processing for a given model results in increasing the context size by the number context with 4 parallel requests will result in an 8K context and additional memory allocation.

The following server settings may be used to adjust how Ollama handles concurrent requests on i

- `OLLAMA_MAX_LOADED_MODELS` - The maximum number of models that can be loaded concurrently provided they fit in available memory. The default is 3 * the number of GPUs or 3 for CPU inference.
- `OLLAMA_NUM_PARALLEL` - The maximum number of parallel requests each model will process at the same time. The default is 1, and will handle 1 request per model at a time.
- `OLLAMA_MAX_QUEUE` - The maximum number of requests Ollama will queue when busy before rejecting additional requests. The default is 512

Note: Windows with Radeon GPUs currently default to 1 model maximum due to limitations in ROCm v5.7 for available VRAM reporting. Once ROCm v6.2 is available, Windows Radeon will follow the defaults above. You may enable concurrent model loads on Radeon on Windows, but ensure you don't load more models than will fit into your GPUs VRAM.

qwen2.5:7b-instruct-q4_K_M

ollama run qwen2.5:7b-instruct...

↓ 12.5M Downloads

🕒 Updated 11 months ago

Qwen2.5 models are pretrained on Alibaba's latest large-scale dataset, encompassing up to 18 trillion tokens. The model supports up to 128K tokens and has multilingual support.

tools

0.5b

1.5b

3b

7b

14b

32b

72b

Updated 11 months ago		845dbda0ea48 · 4.7GB
model	arch <code>qwen2</code> · parameters <code>7.62B</code> · quantization <code>Q4_K_M</code>	4.7GB
system	You are Qwen, created by Alibaba Cloud. You are a helpful ass...	68B
license	Apache License Version 2.0, January 2004 http://www.apache.or...	11kB
template	{{- if .Messages }} {{- if or .System .Tools }}< im_start >sy...	1.5kB

06 项目实践

为什么生产环境首选 vLLM

vLLM 是当前大模型服务化部署的事实标准，尤其适合高并发、低延迟的线上场景。

特性	vLLM
吞吐量 (Throughput)	极高 (PagedAttention)
首 Token 延迟 (TTFT)	极低 (Chunked Prefill)
KV Cache 效率	PagedAttention (显存利用率 ↑ 70%)
长上下文支持	卓越 (128K+)
多轮对话优化	Prefix Caching (复用历史KV)
生产功能	完整 (批处理、优先级调度、监控)

如何保证 P99延迟100ms，5秒热更新？

P99 延迟：99% 的请求响应时间都低于该值。例如 P99=82ms

实现路径：

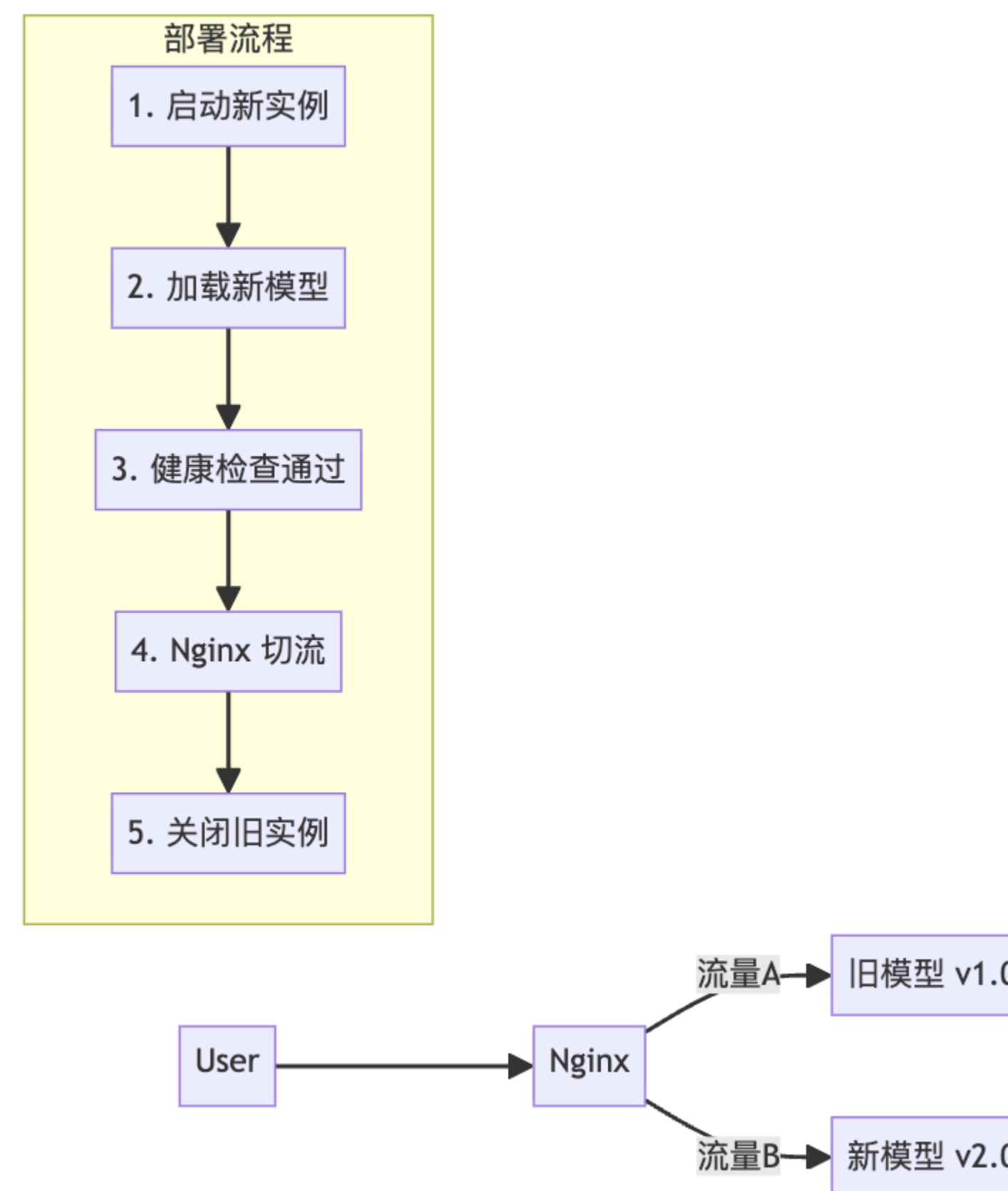
1. 模型轻量化：使用 GPTQ-Int4 量化，将 Qwen-7B 模型从 14GB 压缩至 ~4GB。
2. 优化 Prefill 阶段：启用 `enable_chunked_prefill`
3. 优化 KV Cache：启用 `enable_prefix_caching`
4. 调整批处理参数：设置合理的 `max_num_batched_tokens`，平衡延迟与吞吐。

如何保证 P99延迟100ms，5秒热更新？

热更新：如何实现 5秒内模型切换？

vLLM 原生不支持热更新，需结合外部服务发现与负载均衡。

实现方案：双实例蓝绿部署 + Nginx 路由



端到端部署：构建可上线的 FastAPI 服务

```
from fastapi import FastAPI, HTTPException
from vllm import LLM, SamplingParams
import asyncio

app = FastAPI()

# 全局LLM实例
llm = LLM(
    model="Qwen/Qwen2.5-7B-Instruct-GPTQ-Int4",
    quantization="GPTQ",
    enable_prefix_caching=True,
    enable_chunked_prefill=True,
    max_num_batched_tokens=2048,
    gpu_memory_utilization=0.85,
    dtype='half'
)

# 采样参数
sampling_params = SamplingParams(temperature=0.7,
top_p=0.9, max_tokens=512)

@app.post("/predict")
async def predict(request: dict):
    try:
        # 异步推理
        result = await
llm.generate.async_request(request["text"],
sampling_params)
        return {"intent": result.outputs[0].text}
    except Exception as e:
        raise HTTPException(status_code=500,
detail=str(e))

@app.get("/health")
def health_check():
    return {"status": "healthy", "model": "qwen-7b-
int4-gptq"}
```


性能验证与压测

启动服务

```
docker build -t intent-vllm .  
docker run -p 8000:8000 --gpus all  
intent-vllm
```

压测验证 P99:

```
ab -n 1000 -c 50 -m POST -t 60 -d  
' {"text": "我要退票"}'  
http://localhost:8000/predict
```


THANKS

 极客时间 | 训练营